Name: Shayan Umar

Designation: AI intern

Department: Artificial Intelligence

Date: 8-8-2025

Table of Contents

1. Theoretical Understanding:

In **Week-05**, Sir Asad Ahmed assigned us daily tasks, for **Week05 - Day 01** we were assigned 2 task one was implementing ANNs architecture from scratch only using python and numpy.

I implemented weights initialization, forward propagation, backward propagation, activation functions and Loss functions from scratch.

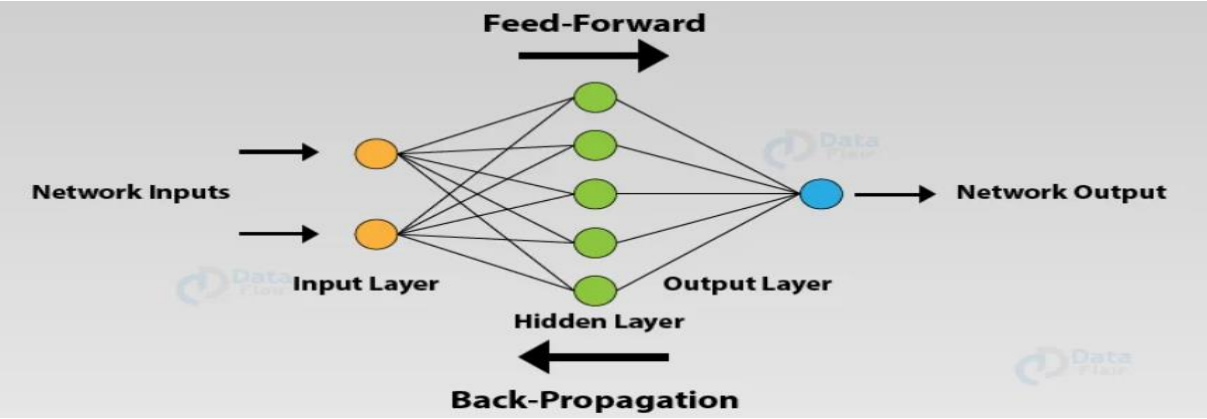
2nd task was to apply the custom ANN to a dataset or use keras.Sequential().

1.1 Artificial Neural Networks (ANNs):

An **Artificial Neural Network (ANN)** is a computational model designed to recognize patterns. It is inspired by the structure of the human brain and consists of layers of **neurons** (also called nodes or units), which are connected by **weights**.

At a high level, an ANN:

- Takes input data,
- Processes it through layers,
- Produces an output (e.g., a prediction or classification).



1. Structure of an ANN

An ANN typically consists of three types of layers:

a. Input Layer

- This layer receives the initial data (e.g., pixel values of an image).

- It does not perform any computations.
- Each neuron in this layer represents one feature of the input data.

b. Hidden Layers

- These layers perform computations on the input.
- The number of hidden layers and neurons per layer can vary.
- Each neuron applies a weighted sum and passes it through an **activation function**.

c. Output Layer

- This layer provides the final result (e.g., a class label or a value).
- The number of neurons in this layer depends on the task:
    - For binary classification: 1 neuron (e.g., spam or not spam).
    - For multi-class classification: one neuron per class.

---

2. How a Neuron Works

Each neuron performs the following operation:

$z = w1*x1 + w2*x2 + ... + wn*xn + b$

$a = activation(z)$

Where:

- x1, x2, ..., xn are inputs
- w1, w2, ..., wn are weights
- b is the bias term
- z is the weighted sum
- a is the output of the activation function

---

3. Activation Functions

An activation function decides whether a neuron should "fire" or not. Common activation functions include:

- **ReLU (Rectified Linear Unit)**:
- $f(x) = \max(0, x)$

Used widely in hidden layers.

- **Sigmoid**:
- $f(x) = 1 / (1 + \exp(-x))$

Squashes values between 0 and 1.

- **Tanh**:
  Similar to sigmoid but outputs values between -1 and 1.
- **Softmax**:
  Used in the output layer for multi-class classification to convert outputs into probabilities.

---

4. Forward Propagation

Forward propagation is the process of:

- Passing the input through the network layer by layer,
- Applying weights and biases,
- Using activation functions,
- Producing the output.

Each layer transforms the data and passes it to the next layer.

---

5. Loss Function

Once we get the output, we compare it with the **true value** using a **loss function** (also called cost function).

Examples:

- **Mean Squared Error**: for regression problems.
- **Cross-Entropy Loss**: for classification problems.

The goal of training is to minimize this loss.

---

6. Backpropagation and Learning

After calculating the loss, the ANN needs to learn from it. This is where **backpropagation** comes in.

- Backpropagation calculates the **gradient** of the loss with respect to each weight.
- These gradients show how much each weight contributes to the error.
- Then, using an **optimizer** (like gradient descent), we **update** the weights in the direction that reduces the loss.

This process is repeated over many **epochs** (passes over the full dataset).

---

7. Training Process Summary

1. **Initialize** weights and biases randomly.
2. **Forward pass**: compute predictions.
3. **Calculate loss**: measure error.
4. **Backpropagation**: compute gradients.
5. **Update weights**: using optimizer.
6. Repeat steps 2–5 for many epochs.

---

8. When to Use ANNs

ANNs are useful when:

- Data is complex and high-dimensional.
- You want to perform tasks like:
    - Image classification
    - Sentiment analysis
    - Forecasting
    - Fraud detection

---

9. Strengths of ANNs

- Capable of learning complex, non-linear relationships.

- Can handle large amounts of data.
- Adaptable to different types of problems (classification, regression, etc.).

---

10. Weaknesses of ANNs

- Require large amounts of data to perform well.
- Training can be computationally expensive.
- Often considered "black boxes" — it's hard to understand how exactly decisions are made.

1.2 Convolutional Neural Networks (CNNs):

Week 5-Day 2 task was to understand and build CNNs architecture from scratch the components include Convolution layer, Pooling Layers, Fully Connected Layer, activation functions, loss function, paddings and Strides.

One task was to build all these components from scratch and to train a model using these components on mnist dataset.

What is a Convolutional Neural Network (CNN)?

A **Convolutional Neural Network (CNN)** is a specialized type of neural network that is **very effective for image-related tasks**, such as:

- Image classification (e.g., MNIST digits, cats vs. dogs)
- Object detection (e.g., detecting faces)
- Image segmentation (e.g., identifying each pixel's class)

It is designed to process **grid-like data** such as 2D images using **convolution operations**.

Why Not Just Use a Regular ANN for Images?

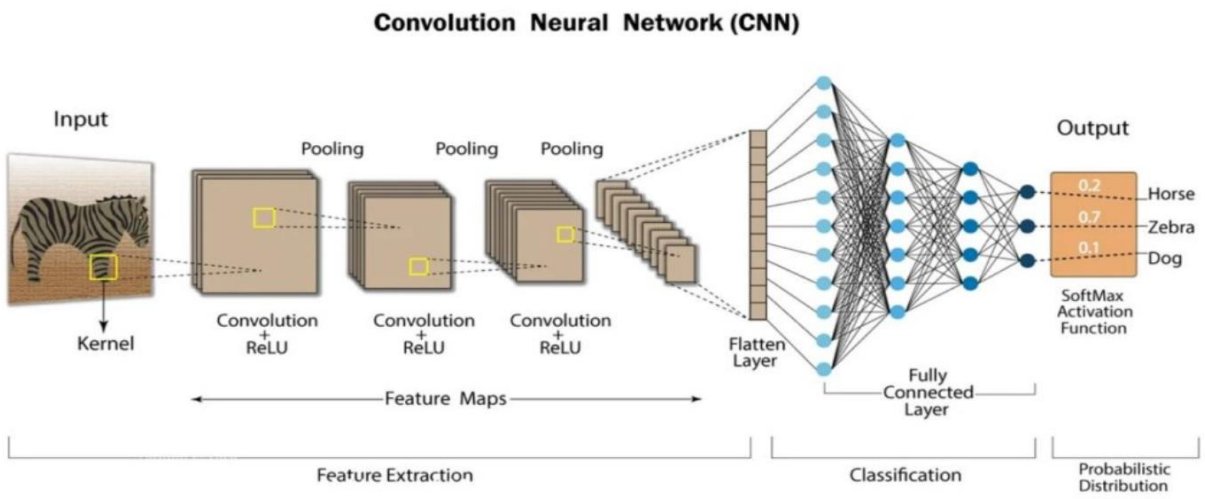
Images can have **thousands of pixels**. For example, a 28x28 grayscale image has 784 features. A color image of size 256x256x3 has over 196,000 inputs.

Using a fully connected ANN for this would:

- Require too many parameters
- Lose spatial information (like edges, shapes)
- Be computationally expensive and prone to overfitting

CNNs solve these problems by:

- Using **convolutional layers** that focus on small regions at a time
- **Sharing weights** (fewer parameters)
- Preserving spatial relationships in data



Key Components of a CNN

1. Convolutional Layer

This is the core layer in a CNN.

- It applies **filters (also called kernels)** to the input image.
- Each filter detects certain features like **edges, corners, textures**.
- The filter slides across the image and produces a **feature map**.

Mathematically:

output = sum(region * filter) + bias

Parameters:

- **Filter size** (e.g., 3x3, 5x5)
- **Stride** (how far the filter moves each time)
- **Padding** (to control the output size)

2. Activation Function (usually ReLU)

After convolution, we apply an **activation function**, commonly ReLU:

f(x) = max(0, x)

This introduces **non-linearity**, allowing the network to learn complex patterns.

3. Pooling Layer (e.g., Max Pooling)

Pooling reduces the spatial size of feature maps:

- **Max Pooling**: takes the maximum value in a window (e.g., 2x2).

- **Average Pooling**: takes the average instead.

Purpose:

- Reduces computation
- Controls overfitting
- Makes feature detection more robust to small shifts

4. Flatten Layer

After multiple convolution + pooling layers, the final feature maps are **flattened** into a 1D vector so they can be passed to fully connected (dense) layers.

5. Fully Connected (Dense) Layer

- Acts as the decision-making part.
- Typically comes near the end.
- Maps the high-level features to final class scores.

6. Output Layer

- Usually uses **Softmax** for classification problems.
- Outputs a probability distribution over classes.

How a CNN Works (Step-by-Step)

Let's say we're using a CNN to classify handwritten digits (MNIST):

1. Input: 28x28 grayscale image.
2. **Convolution Layer**:
    - Applies several 3x3 filters to the image.
    - Produces several 26x26 feature maps.
3. **ReLU Activation**:
    - Converts negative values in the feature maps to zero.
4. **Max Pooling**:
    - Reduces each feature map to 13x13 by taking max values in 2x2 windows.
5. **More Conv + Pool Layers**:

    - Deeper layers learn more abstract patterns (e.g., shapes).
6. **Flatten**:
    - Convert final feature maps into a 1D vector.
7. **Fully Connected Layer**:
    - Learns to map features to class scores.
8. **Softmax Output**:
    - Produces probabilities for digits 0 through 9.

Key Concepts That Make CNNs Powerful

Concept	**Why It Matters**
Local Connectivity	Focuses on small parts of the image at a time
Weight Sharing	Same filter used across the entire image
Translation Invariance	Can detect patterns regardless of position
Deep Hierarchies	Lower layers learn edges, upper layers learn shapes

---

CNN Architecture Example (Simplified)

Input (28x28)

→ Conv Layer (3x3x32 filters)

→ ReLU

→ MaxPool (2x2)

→ Conv Layer (3x3x64 filters)

→ ReLU

→ MaxPool (2x2)

→ Flatten

→ Dense (128 units)

→ ReLU

→ Dense (10 units)

→ Softmax

Advantages of CNNs

- Very good at image and video processing
- Require fewer parameters than fully connected ANNs
- Capture spatial features and hierarchies
- Generalize well with enough training data

Common Applications

- Handwriting and digit recognition (like MNIST)
- Medical image diagnosis
- Face recognition
- Self-driving cars (detecting lanes, signs)
- Object tracking and recognition

1.3 Recurrent Neural Networks (RNNs):

Recurrent Neural Networks (RNNs)

Week 5 – Day 3 task was to understand and build RNN architecture from scratch. Components included:

- Recurrent layers
- Hidden state updates
- Activation functions (e.g., Tanh)
- Loss functions (e.g., cross-entropy)
- Unrolling through time (Backpropagation Through Time - BPTT)
- Training a model on sequence data (e.g., text, time series, etc.)

What is a Recurrent Neural Network (RNN)?

A **Recurrent Neural Network (RNN)** is a type of neural network designed for **sequential data**. This includes:

- Time series (stock prices, sensor data)
- Text and language (natural language processing)
- Audio and speech data

Unlike regular feedforward neural networks, RNNs have the ability to **maintain memory of past inputs** by using loops in their architecture. This allows them to capture patterns and dependencies over time.

Why Not Use a Regular ANN for Sequence Tasks?

Standard ANNs process all inputs independently, which is not ideal for sequences where order and context matter.

For example:

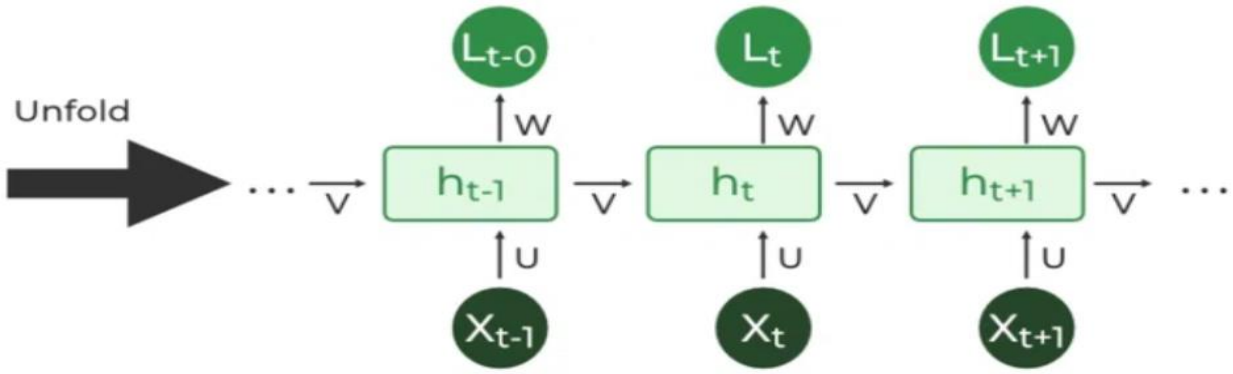
- In text, “not good” and “good” have opposite meanings, but a standard ANN may not recognize the difference without order awareness.
- In time series, predicting the next value requires knowledge of previous ones.

Using a standard ANN for sequences would:

- Require fixed-length input
- Lose temporal relationships
- Ignore sequential structure

RNNs solve this by:

- Retaining memory via hidden states
- Using the same weights at each time step
- Capturing dependencies between previous and current inputs



Key Components of an RNN

1. Recurrent Layer

This is the heart of an RNN.

At each time step t, the RNN takes the current input x_t and the previous hidden state h_{t-1} to compute a new hidden state h_t:

h_t = tanh(Wx * x_t + Wh * h_{t-1} + b)

Where:

- Wx is the input weight matrix
- Wh is the recurrent weight matrix
- b is the bias
- tanh is the activation function

The same weights are used at every time step, which enables generalization across sequence lengths.

---

2. Activation Function

- Typically, **tanh** or **ReLU** is used.
- tanh maps values between -1 and 1 and is good for maintaining signal flow through time.
- Sometimes **sigmoid** is used for gate mechanisms in LSTMs or GRUs.

---

3. Output Layer

- A **fully connected layer** is usually added at the end of the RNN.
- Often followed by a **softmax** function to generate probability distributions for classification tasks:

softmax(x_i) = e^(x_i) / sum(e^(x_j)) for all j

---

4. Loss Function

- For classification problems: **categorical cross-entropy**
- For regression problems: **mean squared error (MSE)**
- Loss is typically calculated at every time step or only at the last one depending on the task.

How RNNs Work (Step-by-Step for a Text Classification Task)

Example: Sentiment analysis on a movie review

1. **Input**: A sentence is tokenized and converted into a sequence of word vectors.
2. **Recurrent Layer**: Each word vector is passed one at a time into the RNN.
3. **Hidden State Update**: At each step, the hidden state is updated based on the current word and previous state.
4. **Final Hidden State**: The last hidden state summarizes the entire sequence.
5. **Fully Connected Layer**: The final hidden state is passed to a dense layer.

6. **Softmax Output**: Produces class probabilities (e.g., Positive, Negative).

Key Concepts That Make RNNs Effective

Concept	Description
Sequence Awareness	Retains the order of inputs
Shared Weights	Same parameters used across all time steps
Memory Retention	Hidden state passes information across time steps
Contextual Learning	Learns dependencies in sequences (e.g., grammar, trends)

RNN Architecture Example (Simplified)

Input (e.g., sentence of 10 words)
→ **Embedding Layer**
→ **RNN Layer (hidden size = 128)**
→ **Final Hidden State**
→ **Dense Layer (64 units)**
→ **ReLU Activation**
→ **Dense Layer (2 units)**
→ **Softmax Output (Positive / Negative)**

Advantages of RNNs

- Handle variable-length input sequences
- Capture temporal or sequential dependencies
- Useful for a wide range of applications involving time or order

---

Limitations of Vanilla RNNs

- **Vanishing Gradient Problem**: Gradients become too small to update weights properly for long sequences
- **Exploding Gradients**: Gradients become too large, making training unstable

- **Difficulty with Long-Term Dependencies**: Vanilla RNNs struggle to remember inputs that are far apart in a sequence

---

Solutions to RNN Limitations

To address the problems of vanilla RNNs, two popular advanced variants were developed:

1. LSTM (Long Short-Term Memory)

- Introduces memory cells and gates (input, forget, output)
- Designed to remember information over long time periods

2. GRU (Gated Recurrent Unit)

- A simplified version of LSTM
- Fewer parameters and often similar performance

Common Applications

- Text classification (e.g., spam detection, sentiment analysis)
- Language modeling and generation
- Speech recognition
- Time series forecasting (e.g., stock market, weather)
- Machine translation
- Video captioning

---

2. Project Work:

2.1 Artificial Neural Networks (ANNs):

Task 1: Building an ANN from Scratch

- **Objective:** Implement an ANN without using any high-level deep learning frameworks to understand the inner workings of the model.
- **Steps Involved:**
    1. **Initializing Parameters:**
        - Defined the weights and biases for each layer manually.
        - Used small random values for weights initialization to avoid symmetry issues.
    2. **Forward Propagation:**
        - Implemented linear transformations ($Z = W \cdot X + b$) for each layer.
        - Applied activation functions (ReLU for hidden layers, Sigmoid for output layer).
    3. **Compute Loss:**
        - Used **Binary Cross-Entropy Loss** to measure prediction error.
    4. **Backward Propagation:**
        - Calculated derivatives of loss with respect to weights and biases.
        - Applied the chain rule to propagate gradients backward through the network.
    5. **Parameter Updates:**
        - Updated weights and biases using Gradient Descent.
    6. **Activation Functions Implemented:**
        - **ReLU (Rectified Linear Unit):** For hidden layers to introduce non-linearity.
        - **Sigmoid:** For binary classification output.

Task 2: ANN using Keras Sequential on Churn Dataset

As I have previously worked on Churn Dataset so I did not handle the imbalanced this time that's why the results might be a bit not good.

- **Objective:** Apply an ANN using the **Keras Sequential API** for predicting customer churn.
- **Dataset Used:** Churn Dataset (classification problem).
- **Implementation Steps:**
    1. **Data Preprocessing:**
        - Handled missing values.
        - Encoded categorical variables using One-Hot Encoding.
        - Scaled numerical features for faster convergence.
    2. **Model Architecture:**
        - Used the **Sequential API** from Keras.
        - Added Dense layers with ReLU activation.
        - Used Sigmoid activation in the output layer.
    3. **Model Compilation:**
        - Loss function: Binary Cross-Entropy.
        - Optimizer: Adam.
        - Evaluation Metric: Accuracy.
    4. **Model Training & Evaluation:**
        - Trained the ANN on the processed dataset.
        - Evaluated performance on the test set to measure accuracy and loss.

2.2 Convolutional Neural Networks (CNNs):

Task 1: Image Loading and Basic Preprocessing

- **Objective:** Understand the fundamentals of CNN operations by implementing core image processing steps from scratch.
- **Steps Involved:**
    1. **Image Loading & Preprocessing:**
        - Loaded an image and converted it to **grayscale** for simplicity.
        - Normalized pixel values for consistent processing.
    2. **Custom Convolution Function (convolve2D):**
        - Implemented a function to slide a kernel/filter over the image and compute the dot product.
        - Supported multiple kernel types for different effects.
    3. **Implemented & Compared Various Kernels:**
        - **Identity Kernel:** Returns the original image without changes.
        - **Sharpening Filter:** Enhances edges and fine details.
        - **Gaussian Blur:** Smooths the image and reduces noise.
        - **Edge Detector (Horizontal & Vertical):** Highlights edges in specific directions.
    4. **Stride Function (Custom):**
        - Implemented control over how far the filter moves during convolution to affect output size and detail capture.
    5. **Padding Function (Custom):**
        - Added zero-padding around the image to preserve spatial dimensions after convolution.

Task 2: Applying Custom Functions on Handwritten Digits (MNIST Dataset)

- **Objective:** Use the custom convolution, stride, and padding functions on real dataset images only a single forward pass just to get the core intuition.

- **Implementation Steps:**
    1. Loaded the **MNIST dataset** of handwritten digits (28x28 grayscale images).
    2. Applied **custom convolution** with different filters to observe effects on digit representation.
    3. Experimented with **different strides and padding** to analyze their influence on feature maps.
    4. Compared outputs of identity, sharpening, blur, and edge detection kernels to understand feature extraction in CNNs.

2.3 Recurrent Neural Networks (RNNs):

Task 1: Understanding and Implementing RNNs from Scratch

- **Objective:** Learn the theoretical foundation of RNNs and implement them manually to understand the underlying mechanics.
- **Key Steps & Concepts:**
    1. **Forward Pass Implementation:**
        - Manually computed the hidden state updates:

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$$

where h_t is the hidden state at time step t.

    2. **Unrolling Through Time:**
        - Represented the RNN as multiple repeated units for each time step, sharing the same parameters.
    3. **Backward Propagation Through Time (BPTT):**
        - Implemented gradient calculations by propagating errors backward through all time steps.
        - Updated weights using gradient descent.
    4. **Hidden States:**

- Designed the RNN to store sequential context in its hidden states, allowing it to model temporal dependencies.

5. **Custom Functions:**
    - Created parameter initialization, forward pass, loss computation, and gradient update functions from scratch.
6. **Vanishing Gradient Problem:**
    - Studied how repeated multiplication of small gradients during BPTT leads to values approaching zero.
    - Understood its effect: RNN struggles to learn long-term dependencies.
    - Discussed mitigation techniques like **LSTM** and **GRU** architectures.

Task 2: Simple RNN with Keras Sequential (IMDB Sentiment Classification)

- **Objective:** Apply RNNs in practice for a binary sentiment classification task.
- **Dataset Used: IMDB Movie Review Dataset** (Positive/Negative sentiment labels).
- **Implementation Steps:**
    1. **Data Preprocessing:**
        - Tokenized text reviews and converted them into integer sequences.
        - Applied **padding** to ensure uniform sequence lengths.
        - Split into training and testing sets.
    2. **Model Architecture:**
        - Used Keras **Sequential API**.
        - Added an **Embedding layer** for dense word representations.
        - Added a **SimpleRNN layer** to process sequential data.
        - **SimpleRNN** was not working so instead I used Birdirectional SimpleRNN and dropout layers.
        - Used a Dense output layer with **sigmoid activation** for binary classification.
    3. **Compilation & Training:**
        - Loss function: Binary Cross-Entropy.

- Optimizer: Adam.
- Metric: Accuracy.
- Trained on the dataset for multiple epochs.

4. **Evaluation:**
    - Evaluated accuracy and loss on the test dataset.
    - Observed the RNN's ability to capture sentiment from sequences.

3. Challenges and Approaches:

1. Artificial Neural Networks (ANNs):

- **Challenge:** Weight initialization was tricky, and implementing **backward propagation** from scratch proved difficult.
- **Approach:** Initially, I referred to online resources to correctly implement backward propagation. I plan to re-implement it entirely from scratch in the future to gain complete command and control over the process.

2. Convolutional Neural Networks (CNNs):

- **Challenge:** Working with images was more complex compared to working with tabular data. Understanding convolution operations and the mathematics behind filters was initially challenging.
- **Approach:** I first solved the convolution mathematics by hand to build a strong conceptual understanding. This helped me confidently translate the process into code and overcome the difficulty.

3. Recurrent Neural Networks (RNNs):

- **Challenge:** This was the most challenging part. While I understood the theoretical intuition, coding the **forward** and **backward propagation** was tough.
- **Approach:** For now, I took assistance from ChatGPT to implement the code but plan to code it independently in the future for mastery.
- **Additional Issue:** When applying RNN to the **IMDB dataset**, the model overfitted significantly.
- **Solution:** With guidance from my lead, I reduced overfitting to some extent by using **Bidirectional RNN**, which improved performance.

4. Result and Analysis:

1. Artificial Neural Networks (ANNs):

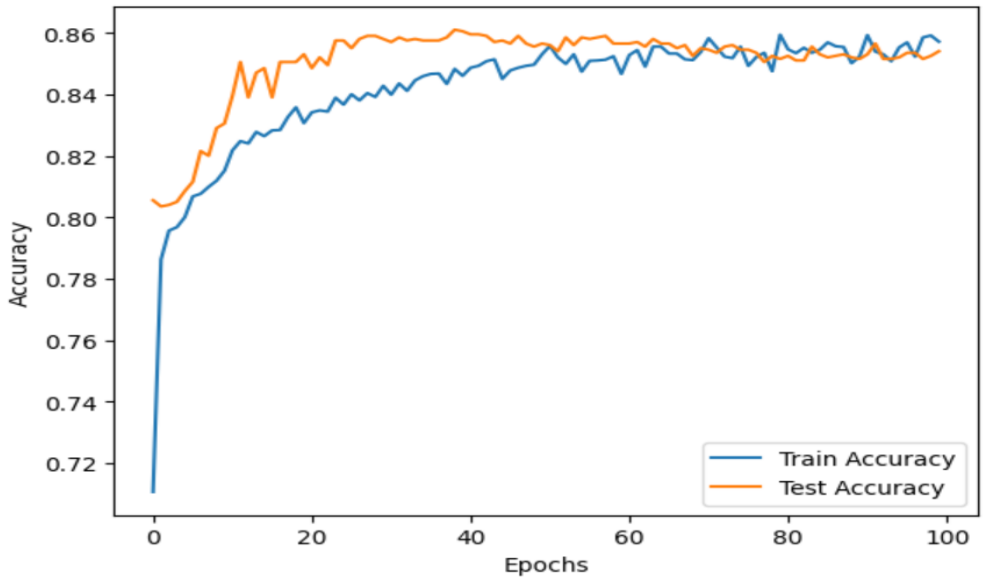
Task 1 results:

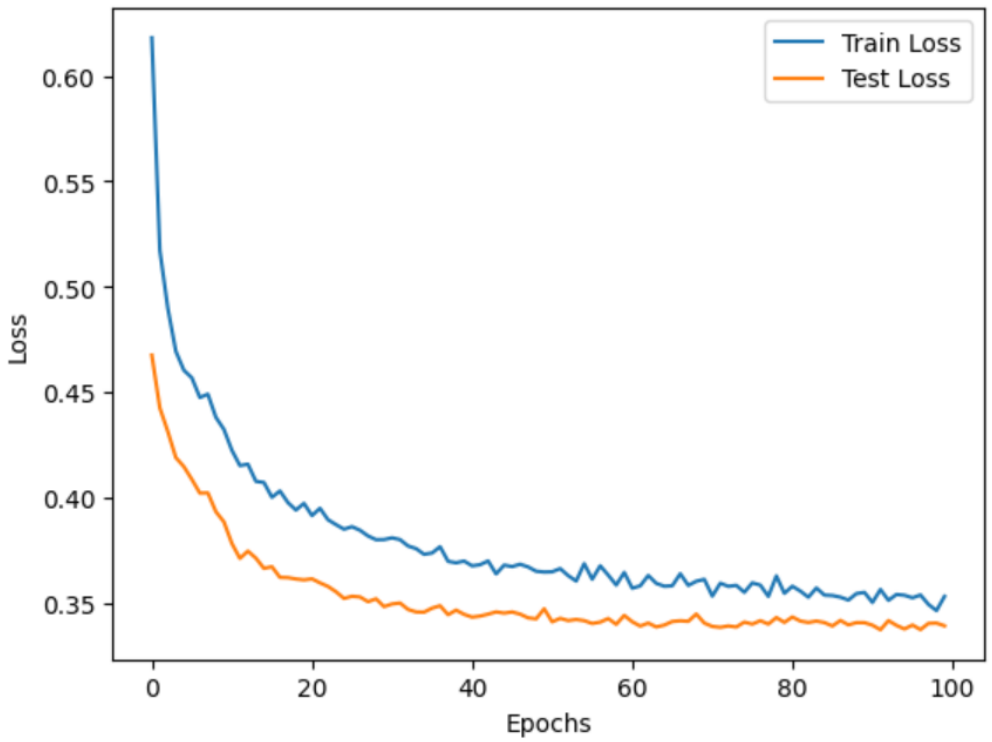
X = np.array([[0, 0, 1, 1], [0, 1, 0, 1]])

Y = np.array([[0, 1, 1, 0]])

```
Epoch 0: Cost = 0.6931479921442534
Epoch 100: Cost = 0.6931375555725924
Epoch 200: Cost = 0.6930925315659666
Epoch 300: Cost = 0.6929143200119674
Epoch 400: Cost = 0.6921961798398933
Epoch 500: Cost = 0.689229812165057
Epoch 600: Cost = 0.6778185063183944
Epoch 700: Cost = 0.6414940013361378
Epoch 800: Cost = 0.5761203208044623
Epoch 900: Cost = 0.5244872666172293
Epoch 1000: Cost = 0.49904565850387417
Epoch 1100: Cost = 0.48009165061515496
Epoch 1200: Cost = 0.41605274196374153
Epoch 1300: Cost = 0.3014650213708051
Epoch 1400: Cost = 0.20893788378825892
Epoch 1500: Cost = 0.14647600349812526
Epoch 1600: Cost = 0.1074237685526931
Epoch 1700: Cost = 0.08224390527374116
Epoch 1800: Cost = 0.06494777276276112
Epoch 1900: Cost = 0.05315453632347123
Predictions: [[0 1 1 0]]
```

Task 2 Results:





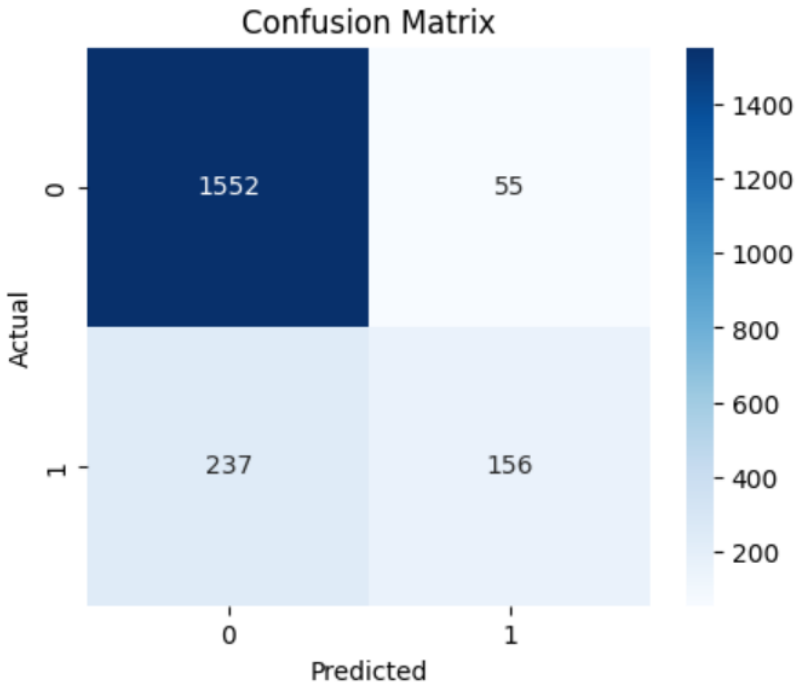
accuracy: 0.8535 - **loss:** 0.3636 - **val_accuracy:** 0.8540 - **val_loss:** 0.3392

Test Accuracy: 85.40%

Test Loss: 0.3392

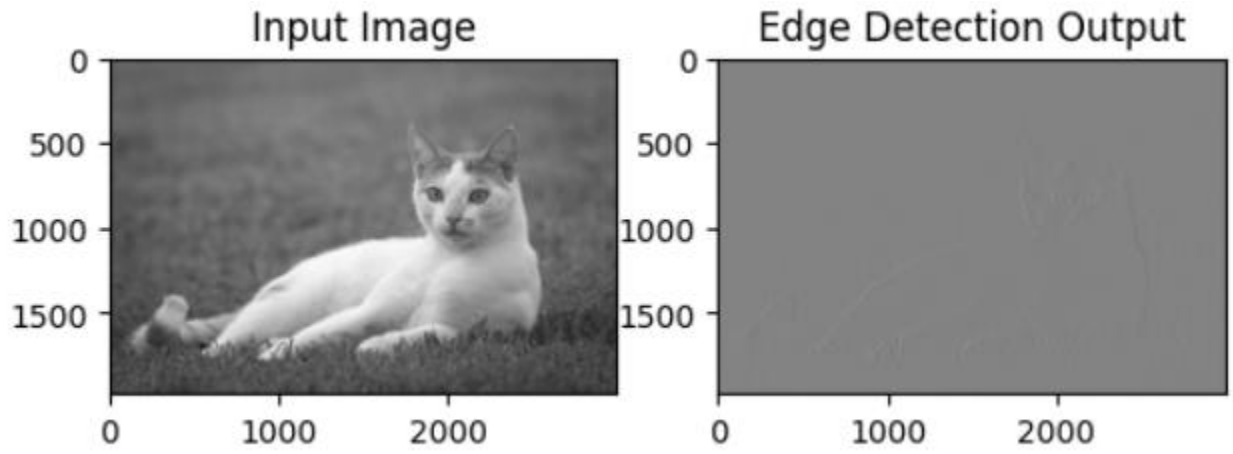
ROC AUC Score: 0.8578

	precision	recall	f1-score	support
0	0.87	0.97	0.91	1607
1	0.74	0.40	0.52	393
accuracy			0.85	2000
macro avg	0.80	0.68	0.72	2000
weighted avg	0.84	0.85	0.84	2000



2. Convolutional Neural Networks (CNNs):

Task 1:



Task 2:

As task 2 was to only do a single forward pass so no confusion matrix etc.

```python
def convolve2D(image,kernal,padding=0,stride=1):
    h, w = image.shape
    kh, kw = kernal.shape
    oh = (h - kh) // stride + 1
    ow = (w - kw) // stride + 1
    output = np.zeros((oh,ow))

    for i in range(0,oh):
      for j in range(0,ow):
        region = image[i*stride : i*stride + kh, j*stride : j*stride + kw]
        output[i,j] = np.sum(region * kernal)


    return output
```

```python
def max_pool(image,size=2,stride=2):
  h, w = image.shape
  oh = (h - size) // stride + 1
  ow = (w - size) // stride + 1
  output = np.zeros((oh,ow))

  for i in range(0,oh):
    for j in range(0,ow):
      region = image[i*stride:i*stride + size , j*stride:j*stride + size]
      output[i,j] = np.max(region)


  return output
```

```python
def fully_connected_layer(pool_output, n_output=10):
    flatten = np.array([item for row in pool_output for item in row])

    input_size = flatten.shape[0]

    np.random.seed(42)

    weights = np.random.randn(n_output,input_size) * 0.01
    bias = np.random.randn(n_output) * 0.01

    output = np.dot(weights,flatten) + bias

    return output
```

```
Final probabilities: [0.086623   0.11591289 0.09584663 0.10796968 0.09603588 0.11635424
 0.10181901 0.08780298 0.11098158 0.08065411]
True label (one-hot): [0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
Cross-entropy loss: 2.1511159161864035
Predicted class: 5
True class: 5
✅ Correctly classified!
```

```
✅ Total Correct: 18 / 100
📊 Accuracy: 18.00%
```

Because it was only a single forward pass and no backward pass or training but still 18% accuracy on test data and in task 2 custom code was used for the forward pass.

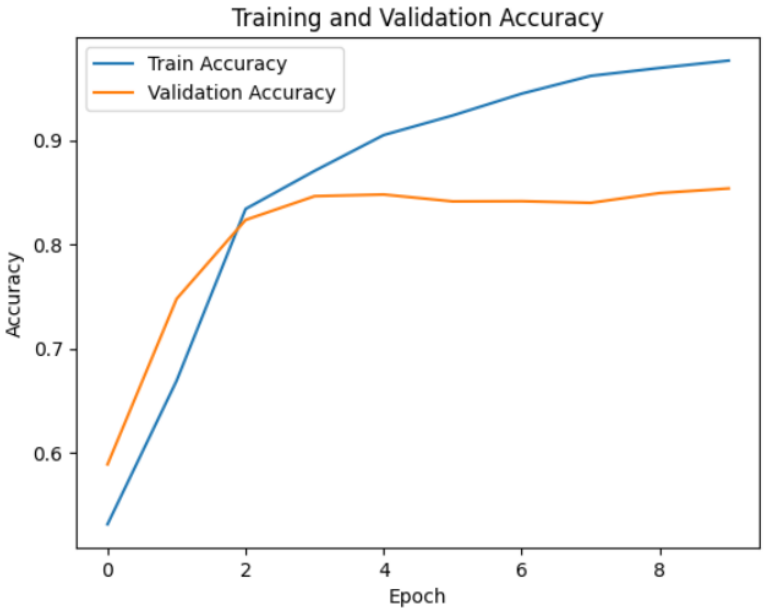
3. Recurrent Neural Networks (RNNs):

Task 1: RNN from Scratch

```
Training sequence:
Inputs:  ['a', 'b', 'c', 'd']
Targets: ['b', 'c', 'd', 'e']

Epoch 0, Loss: 6.4380
Epoch 100, Loss: 0.0798
Epoch 200, Loss: 0.0267
Epoch 300, Loss: 0.0157
Epoch 400, Loss: 0.0111
Epoch 500, Loss: 0.0085
Epoch 600, Loss: 0.0069
Epoch 700, Loss: 0.0058
Epoch 800, Loss: 0.0050
Epoch 900, Loss: 0.0044

Training completed!
```

```
=== Testing the RNN ===

Testing predictions:
Input: 'a' -> Predicted: 'b' (confidence: 0.999)
Input: 'b' -> Predicted: 'c' (confidence: 0.999)
Input: 'c' -> Predicted: 'd' (confidence: 0.999)
Input: 'd' -> Predicted: 'e' (confidence: 0.999)

=== Generating new sequence ===
Starting with 'a', generating 4 characters:
Generated: abcde
```

Task 2: Applying RNN on IMDB Dataset

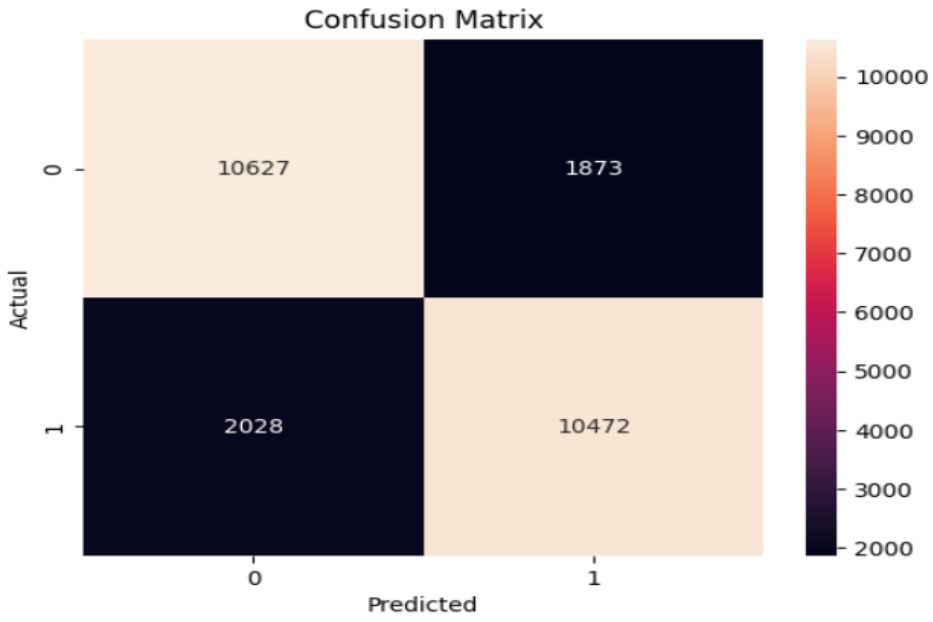


accuracy: 0.9790 - loss: 0.0730 - val_accuracy: 0.8536 - val_loss: 0.4527

Accuracy on Test Data: 90%

AUC-SCORE --> 84.39

	precision	recall	f1-score	support
Negative	0.84	0.85	0.84	12500
Positive	0.85	0.84	0.84	12500
accuracy			0.84	25000
macro avg	0.84	0.84	0.84	25000
weighted avg	0.84	0.84	0.84	25000



5. Conclusion:

Through implementing ANNs, CNNs, and RNNs from both scratch and using high-level APIs, I strengthened my understanding of neural network architectures, forward and backward propagation, and real-world applications.

Through out the week I understand about how neural networks make predictions and how weights and biases are calculated + how loss is minimized using gradient descent or any other optimization algorithm using backpropagation.

While challenges like weight initialization, coding complex propagations, and overfitting required external help, they also highlighted areas for improvement. Moving forward, I plan to re-implement ANN and RNN from scratch to gain full control and continue refining my skills in handling complex datasets and architectures.

6. References:

- https://www.youtube.com/
- https://www.youtube.com/@campusx-official
- https://keras.io/api/layers/convolution_layers/
- https://www.kaggle.com/datasets